

API Dependabot Project Documentation

AI Engineering Capstone | Final implementation and evaluation report | 28 September 2026

Executive summary

API Dependabot is a capstone prototype for identifying potential Java consumer impact when an OpenAPI 3.x provider contract changes. It combines deterministic contract comparison, repository search and lexical retrieval, and two answer strategies: Vanilla RAG and a ReAct tool-using agent.

The latest six-case evaluation passed all 12 approach-case executions. Vanilla RAG passed 6/6, and ReAct passed 6/6 after a grounding instruction change. This is development-set evidence, not a final architecture decision: the set is small, there is no held-out assessment, and the LLM judge needs human review.

Problem and scope

A changed API specification does not by itself tell a consumer team whether its Java code uses the affected operation or property. This project compares old and new OpenAPI files and connects changes to evidence in a Maven/Spring Boot consumer repository.

- Supported: OpenAPI 3.x, Java/Spring Boot, Maven, local repositories, and GitHub repository snapshots.
- Output: deterministic contract changes, relevant Java evidence, and an evidence-cited migration assessment.
- Not implemented: hosted HTTP service, automatic code edits, Docker-isolated consumer tests, GitHub pull request creation, auto-merge, or deployment.

This is a review assistant prototype, not a production Dependabot service.

System design and implementation

Contract and repository analysis

The Java application parses and compares two OpenAPI documents through a deterministic diff layer. Known classification gaps are surfaced as review notes rather than treated as proof of safety. The repository service uses either a local folder or shallow GitHub clone. Private repository credentials are read from GITHUB_TOKEN and are not embedded in the repository URL.

Retrieval and answers

Java source, tests, README, Maven configuration, and the contract diff are split into bounded chunks. A deterministic BM25-style lexical ranker returns relevant chunks with file and line ranges. No vector database is used. Vanilla RAG retrieves once and makes one model request. ReAct can inspect the diff, search Java, read a bounded Java range, and retrieve evidence through read-only, repository-scoped tools; it has an eight-call limit.

Grounding behavior

ReAct instructions require evidence linking a changed operation and field to a request/response call site or mapping before claiming repository-specific impact. If the connection is missing, the answer should state the gap and frame actions conditionally. Both approaches cite evidence identifiers; recommendations remain advisory.

Framework choices

Choice	Reason
Java 21 / Spring Boot / Maven	Matches the user and consumer ecosystem; keeps the CLI and services in one stack.
Spring AI	Provides model integration and tool calling while diff/retrieval remain separate.
OpenAPI parser/diff	Handles specification syntax; fixtures make classification behavior testable.
Promptfoo	Runs both approaches on shared labeled inputs with deterministic and judge assertions.

Evaluation method

The Promptfoo suite contains six development cases, each executed once with Vanilla RAG and once with ReAct. Cases cover a removed response property, removed operation, added operation, newly required request field, renamed path parameter, and Stripe Basil-to-Clover migration. Each answer is checked for a case-specific symbol and graded with the same LLM-as-a-judge rubric for correctness, evidence, and uncertainty.

The application defaults to gpt-5-mini unless APIDEPENDABOT_MODEL overrides it. The saved summary does not record an effective override, and the rubric grader model is not explicitly pinned in the evaluation config; record both explicitly for future comparable runs.

The validation script also runs Maven tests and an optional public GitHub checkout/search smoke check before evaluation. The latest script run reported 26 automated tests passing and the GitHub smoke check passing.

Comparative results

Run	Vanilla RAG	ReAct	Interpretation
Initial six-case run	6/6; judge 1.00; median 14.8 s	5/6; judge 0.92; median 29.5 s	ReAct said sample service/tests needed changes without evidence of an API call.
Rerun after grounding change	6/6; judge 1.00; median 12.4 s	6/6; judge 1.00; median 33.5 s; 42 tool calls	The prior failing case passed; one rerun does not prove general improvement.

In the rerun, both approaches passed all 12 provider-case results. ReAct's median latency was about 2.7 times Vanilla RAG's, and ReAct made 42 tool calls across six answers. Quality, latency, and orchestration cost need assessment on held-out cases and human review before an architecture choice.

Failure analysis and decisions

Unsupported repository impact

On the initial run, ReAct correctly identified that POST /widgets required a new category property. It then claimed the sample Widget model, service, and tests needed changes even though repository evidence did not show that those files sent the affected request. The judge rejected the unsupported impact claim. The system prompt was revised to require a call-site or payload-mapping link before asserting repository impact. The answer passed on the same six-case set. This is a promising fix for the observed case, not proof that all such errors are resolved.

Earlier implementation pivots

The README records implementation failures and pivots: adapting to the diff library's writer/resource behavior on Windows; separately surfacing removed response fields and renamed parameters when library classifications did not; distinguishing identifier references from declarations in lexical search; improving ranking after irrelevant chunks outranked code; and fixing invalid bounded-file-read calls before evaluation.

Known limitations

- Only six curated development cases have been run, with one rerun after a prompt revision; there is no held-out set.
- The judge can miss errors or reward plausible wording. Manual review is required.
- Lexical matching can miss indirect/generated/reflection use and does not trace AST call graphs.
- No code patching, Docker-based test execution, PR creation, HTTP API, auth layer, or deployed service is included.
- The GitHub smoke test verifies shallow checkout and search against Spring Petclinic, not all repository shapes.

Reproduction steps

Install Java 21, Git, and Node.js 22.22.0 or later with npm. From the cloned project root, set model access and run the validation script in PowerShell:

```
$env:OPENAI_API_KEY = "your-api-key"
$env:SPRING_AI_MODEL_CHAT = "openai"
.\scripts\run-capstone-validation.ps1 -GitHubRepository "https://github.com/spring-projects/spring-petclinic"
```

The script builds the JAR, runs Maven verification with the portable Eclipse compiler profile, checks out and searches the optional GitHub repository, evaluates all six shared cases, and writes evaluation/promptfoo/summary.json. Detailed outputs are in evaluation/promptfoo/results.json. A failed assertion returns a nonzero status after the summary is written. Model answers make live provider calls.

Three-minute demo walkthrough

Time	Show and explain
0:00-0:30	State the scope: OpenAPI 3.x changes, Java/Spring impact, advisory CLI.
0:30-1:00	Run the deterministic diff; point out the removed response property.
1:00-1:35	Show Vanilla RAG on the sample question and its source citations.
1:35-2:10	Show ReAct's read-only tool trace and repository evidence.
2:10-2:45	Open the summary; compare paired results and describe the failure/prompt pivot.
2:45-3:00	State limitations and the next step: held-out cases and human review.
Recording tip: use saved evaluation outputs. Rehearse with only the two live demo calls.	